CSCI 49900: Capstone

Group 8: Alex Haskel, James Lacoste, Kailun You

11/04/2026



Overview

- Our project is an interactive platform to demonstrate the mechanics of large language model inference optimization and to allow for the exploration of model behavior through internal interventions.
- The website will allow: Prompt → Two configurable experimental runs → side-by-side outputs + metrics.
- The project requires writing code to optimize a model, and creating an interactive platform in which to test model behavior and performance.



General Progress

- Backend now handles concurrency, returns real latency and throughput metrics, and has more robust error handling.
- CORS (Cross-origin resource sharing) middleware now enabled in API so the browser-based frontend can securely send requests to the FastAPI backend during local development.
- Implemented simple tutorial to explain each sections of the website's function
- Started building the wrapper to allow the python backend to trigger the c++ kernel



Backend Development (Alex)

Concurrency Handling:

- If two users make concurrent requests, and each of these requires making certain modifications on the model, these could conflict with each other.
- Added concurrency handling measures around each generation request to ensure such conflicts cannot occur.
- Now there are locks around the generation process:

  obtain lock → request text generation → other requests wait to obtain lock → first request finished → lock released → next request served.

Concurrency (Alex)

```python
#Concurrency management lock
generation_lock = threading.Lock()

@app.post("/generate", response_model=GenerateResponse)
def generate(request:GenerateRequest):

    prompt = request.prompt
    configuration_a = request.configuration_a
    configuration_b = request.configuration_b

    #### START LOCK HERE #####
    with generation_lock:
        #Generate text using first configuration
        try:
            result_a = run_generation(model, tokenizer,
```

Create shared lock object

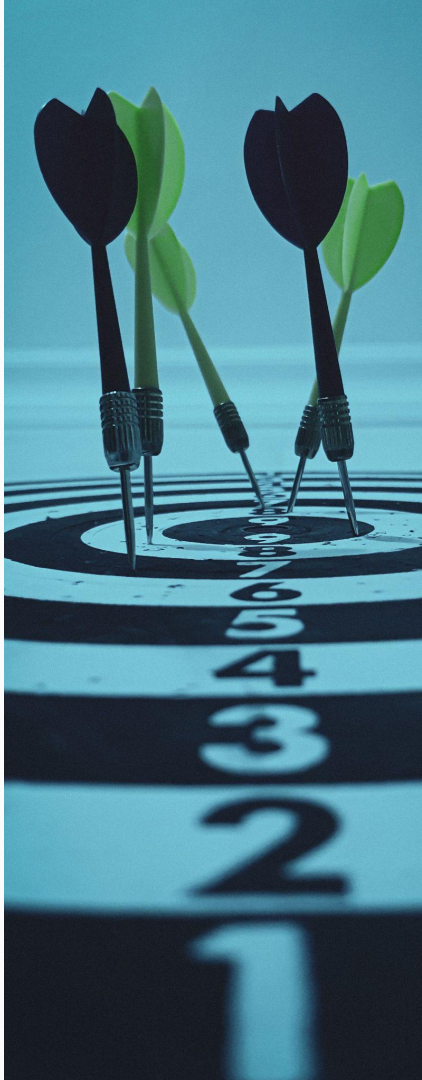
Acquire lock before initiating generation request

If a second thread reaches this section and tries to acquire the lock, it will keep waiting and trying until it is released by previous processes before proceeding.

(Not shown: wrote tests to check this behavior works as intended)

Independent Success and Error States (Alex)

- Users always send two text generation requests to backend: one for ‘configuration A’ and another one for ‘configuration B’.
- Up to this point, any kind of error in either of the requests would halt the entire process and produce an error.
- Changed generation logic so that each configuration can succeed or fail independently.
- This will be especially helpful if we later add things like streaming output.
- Also wrote tests to check independent behavior works



Independent Success and Error States (Alex)

```python
class GenerateResponse(StrictModel):
    condition_a: GenerationSuccess | GenerationError
    condition_b: GenerationSuccess | GenerationError
```

Response schema now allows each configuration to independently be one of two types: success or error

```python
class GenerationSuccess(StrictModel):
    state: Literal["success"]
    text: str
    latency_ms: float
    num_generated_tokens: int
    tokens_per_second: float
```

This is the usual response when things go as expected

```python
class GenerationError(StrictModel):
    state: Literal["error"]
    error_type: str
    message: str
```

We can use this to return useful error information in each response object independently



Adding System Prompts

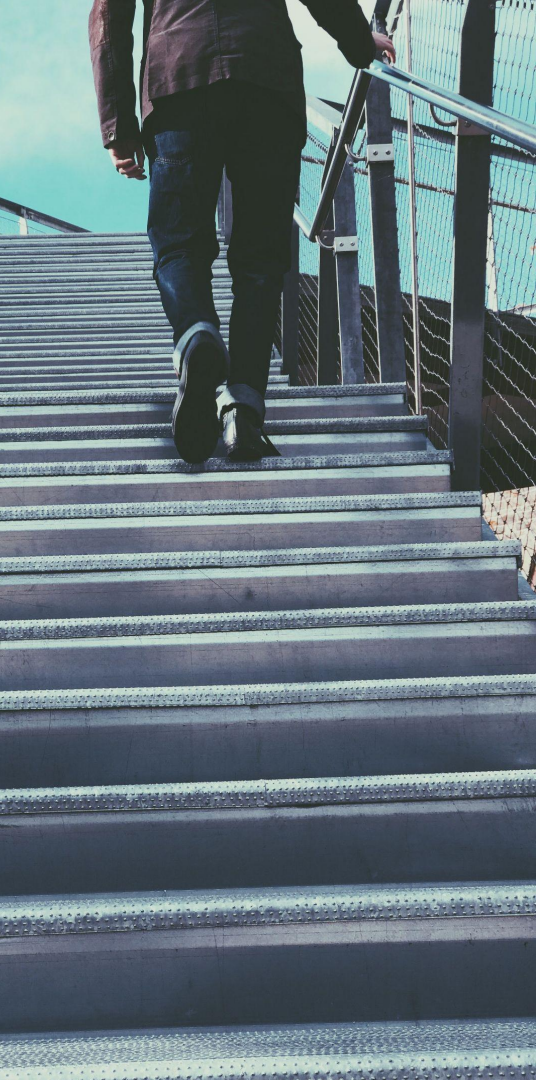
- A system prompt is the initial instructions you give an AI model that sets its behavior, personality, constraints, and role for a conversation

- A system prompt might instruct an AI model to "respond concisely" or "think step-by-step," shaping its approach to answering questions.

- Added the ability to optionally provide separate system prompts to each configuration as another kind of experimental control

```
class ExperimentConfiguration(StrictModel):
    generation: GenerationSettings
    system_prompt: str | None = Field(default=None, max_length=500)
    intervention: InterventionSettings | None = None
```

System prompt is now an optional field in the request object



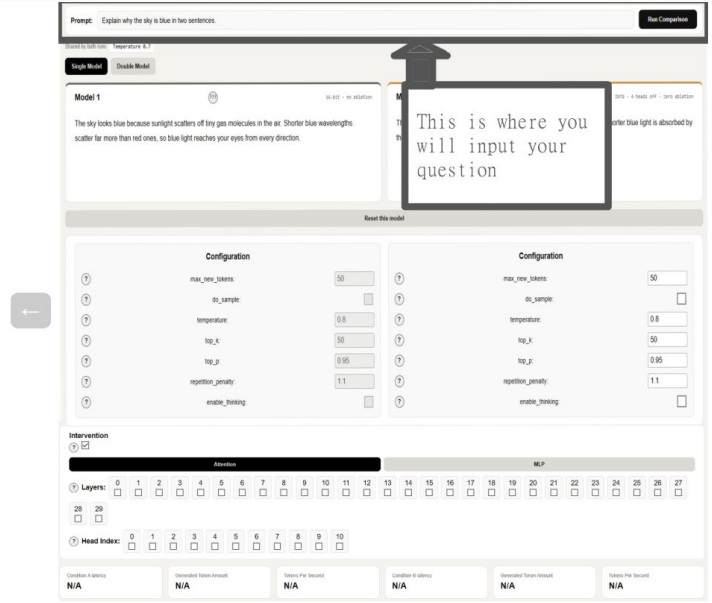
Next Steps (Alex)

- One big change would be to stream output. This would be somewhat involved but would make for a better user experience.
- Related: Make conversations continuous. We currently allow single prompts, but we could allow users to have turn-based conversations.
- Create more elaborate metrics.
- Incorporate custom kernel/quantization type into schemas and rest of the backend.

Frontend Development (Kailun You)

Currently…

- A simple tutorial slides that allows the user to understand what each section would do.
- Question mark that will explain what each input will do to the model and its maximum and minimum amount
- Capped each inputs max and min so inputs won't cause error’s





Frontend Development (Kailun You)

Next objective…

- Implement a second intervention
- Error case implementation
- Reconfigure fetch function
- Display wrong input (maybe)



Building Python/c++ wrapper

- The CUDA kernel, which is written in c++, doesn’t allocate tensors or understand/interact with native PyTorch objects

- There, we need a wrapper to handle those responsibilities

Define the matrix vector multiplication method in c++ to be called from python

```
torch::Tensor int8_gemv(
    const torch::Tensor& x,
    const torch::Tensor& qweight,
    const torch::Tensor& scales,
    const std::optional<torch::Tensor>& bias
)
```

Requirements before the function call:

- x must already be on the GPU
- Its dtype must be FP16 or BF16
- Quantized weight must be int8
- Its shape must be [1, K]
- It must be contiguous

```
check_activation(x);



TORCH_CHECK(
    qweight.scalar_type() == torch::kChar,
    "qweight must be int8"
);
```

Then,

- Select the correct GPU
- Allocate the output tensor

```
const c10::cuda::CUDAGuard guard(x.device());
auto y = torch::empty({1, n_rows}, x.options());
```

This output allocation is performed for each invocation

A future optimized path might try preallocated output buffers or fused operations

Next,

- Use the current CUDA stream
- It is essentially an ordered sequence of GPU operations
- Then launch the kernel

```
const auto stream =
    at::cuda::getCurrentCUDAStream();


int8_gemv_v0_kernel<scalar_t>
    <<<grid_for(n_rows), kThreads, 0, stream>>>(
        x.data_ptr<scalar_t>(),
        qweight.data_ptr<int8_t>(),
        scales.data_ptr<float>(),
        bias.has_value()
            ? bias->data_ptr<float>()
            : nullptr,
        y.data_ptr<scalar_t>(),
        n_rows,
        x.size(1)
    );
```

Next Steps

- Continue with the kernel development by looking into more optimization methods for matrix vector multiplication
- Some examples include: better thread to row mapping, loading multiple weights per instruction, or combining kernel operations
- Once the kernel has been written, try function calls for the kernel through the API